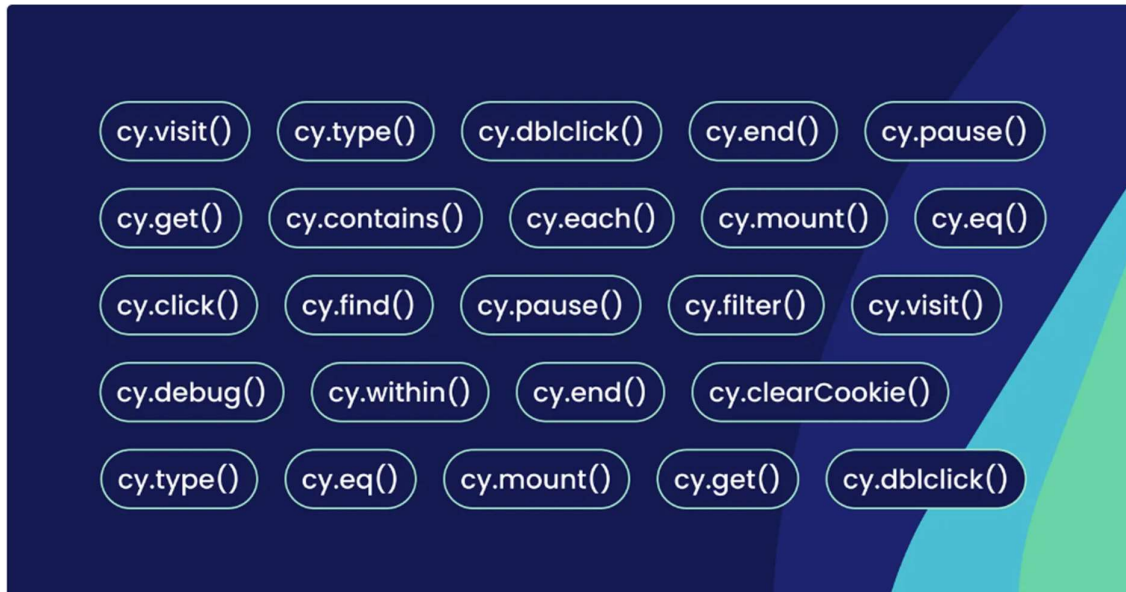


Cypress for Beginners: 5 Commands to Kickstart Your Testing Journey



To any newcomers of testing: Welcome to the exciting world of Cypress! If you're new to web application testing, Cypress is a fantastic framework that makes writing and running tests a breeze. To start your journey, let's explore five essential Cypress commands that will quickly become your testing arsenal.

1. Visit Your Test Subject: [cy.visit\(\)](#)

The first step in any Cypress end-to-end test is to tell it where to start – enter [cy.visit\(\)](#). This command loads the web page you want to interact with and test. Think of it as the starting line of your test race. Whether it's a live website or a simple HTML file on your computer, [cy.visit\(\)](#) is your trusty navigator.

There's more to [cy.visit\(\)](#) than meets the eye. You can customize its behaviour:

Set a default base URL in your [Cypress configuration]
(<https://docs.cypress.io/guides/references/configuration>) to avoid typing the full address every time.

Here's how your tests would look after the [baseUrl](#) is set:

```
cy.visit('/', { timeout: 10000 }) // Add a 10 second timeout
```

By mastering [cy.visit\(\)](#), you'll be able to control the starting point of your tests and ensure your virtual user lands on the right page before you put it through its paces.

Testing Handout - 21

2. Grab Elements on the Page: `cy.get()`

Now that you've landed on your webpage, it's time to interact with it. This is where `cy.get()` shines. It's your way of telling Cypress, "Hey, find this specific element on the page so I can work with it." Think of it like playing a game of "I Spy." You describe what you're looking for, and Cypress locates it for you. Here are the common ways to select elements:

CSS Selectors: These are like the addresses for elements on your page. You can target them by their tag name (e.g., `button`, `div`), class name (e.g., `.my-button`), ID (e.g., `#submit`), or any combination of these. One reliable way to select elements is by using data attributes. These are custom attributes you add to your HTML, like `data-cy="login-button"`. This way, even if your CSS changes, your tests remain stable.

Here's how you'd use `cy.get()` with some selectors:

```
cy.get('button') // Get all buttons on the page
cy.get('.my-button') // Get elements with the class "my-button"
cy.get('#submit') // Get the element with the ID "submit"
cy.get('[data-cy="login-button"]') // Get the element with the data-cy attribute "login-button"
```

`cy.get()` is incredibly versatile. You can also chain it with other commands to perform more specific queries or actions on the elements you find. For example:

```
cy.get('button').click() // Find a button and click it
cy.get('button').first() // Query only the first of all buttons
cy.get('input[name="email"]').type('test@example.com') // Find an input field and type into
```

By mastering `cy.get()`, you'll be able to target and manipulate elements on your web page, making your tests truly interactive.

3. Click Away: `cy.click()`

Once you've used `cy.get()` to grab hold of your target element, it's time to unleash the power of interaction with `cy.click()`. This command does exactly what it says – it simulates a real user clicking on the selected element. Whether it's a button that triggers an action, a link that takes you to a new page, or any other clickable element, `cy.click()` is your virtual fingertip. The basic usage is beautifully simple:

```
cy.get('button').click() // Find a button and click it
cy.get('[data-cy="submit-button"]').click() // Find a button by its data attribute and click it
```

But `cy.click()` has a few tricks up its sleeve:

Multiple Clicks: Want to click multiple elements? Use the `multiple` option:

```
cy.get('button').click({ multiple: true })
```

Testing Handout - 21

Forced Clicks: Sometimes, elements might be visually hidden or obstructed. No problem! `cy.click()` can push through with the `force: true` option:

```
cy.get('.hidden-button').click({ force: true })
```

Click Positions: You can even control where on the element the click happens (top left, center, bottom right, etc.):

```
cy.get('#my-image').click('topRight')
```

With `cy.click()`, you can mimic a wide range of user interactions, from simple button clicks to complex navigation sequences. This makes your Cypress tests feel like a real user is exploring your application, ensuring a robust and comprehensive testing experience.

4. Type Like a User: `.type()`

Let's face it, most web applications involve some form of text input – filling out forms, searching for products, writing comments, and so on. Cypress has you covered for all of these scenarios with the `.type()` command. This command emulates a user typing into a type-able element.

Imagine you have a login form with an email field:

```
cy.get('input[name="email"]').type('test@example.com')
```

This simple command finds the email input field and types "test@example.com" into it. But `.type()` can do much more:

Special Characters: Want to simulate typing symbols or key combinations? Use curly braces to enclose special character sequences:

```
cy.get('input[name="password"]').type('MyS3cretP@ssw0rd{enter}') // Types password and presses enter  
cy.get('textarea').type('{selectAll}{backspace}') // Selects all text and deletes it
```

Delayed Typing: By default, `.type()` types each character every 10ms to simulate how a user might type. You can slow or speed this up by using the `delay` option:

```
cy.get('input[name="username"]').type('johndoe', { delay: 0 }) // Instantly type all characters  
cy.get('input[name="username"]').type('johndoe', { delay: 100 }) // Type each character with a 100ms delay
```

Typing Events: Cypress also triggers events like `keydown`, `keypress`, and `keyup` as it types, making your tests even more accurate.

With `.type()`, you can automate the input of any kind of text, making your Cypress tests comprehensive and user-friendly.



Testing Handout - 21

5. Check Your Work: `.should()`

After you've navigated to your web page, selected elements, clicked buttons, and filled out forms, it's time for the moment of truth: verifying that your application is behaving as expected. That's where `.should()` steps in. This command is your quality control inspector, allowing you to make assertions about the state of your web page.

Think of assertions as questions you ask Cypress about the elements on your page:

"Is this button visible?"

"Does this heading contain the correct text?"

"Does this element have a specific CSS class?"

With `.should()`, you can pose these questions and Cypress will give you a definitive answer – either the assertion passes (the condition is true) or it fails (the condition is not met). Here's the basic structure:

```
cy.get('h1').should('contain', 'Welcome to My website') // Check if the h1 tag contains the text "Welcome to My Website"
```

In this example, we're first selecting the `<h1>` element and then using `.should()` to assert that it contains the specified text. Cypress provides a wide range of built-in assertions you can use:

Existence: `exist`, `not.exist`

Visibility: `be.visible`, `be.hidden`

Content: `contain`, `have.text`, `have.value`

Classes and Attributes: `have.class`, `have.attr`

Length and Count: `have.length`, `have.length.greaterThan`

You can even combine multiple assertions:

```
cy.get('button').should('be.visible').and('have.class', 'active') // Button must be visible and have the "active"
```

`.should()` not only helps you catch bugs but also makes your tests more readable and self-documenting. By clearly stating your expectations, you create tests that are easy to understand and maintain.

Bonus Tip: Pause and Synchronize with `cy.wait()`

While `cy.visit()`, `cy.get()`, `.click()`, `.type()`, and `.should()` are the bread and butter of Cypress, the `cy.wait()` command is your secret weapon for handling the asynchronous nature of web applications.

Many web apps make requests to servers to fetch data or update information. `cy.wait()` can wait for these requests to complete before proceeding:

```
cy.intercept('/api/users').as('getUsers')  
cy.wait('@getUsers') // Wait for the request aliased as 'getUsers' to finish
```

By intercepting requests and waiting for their responses, you can ensure your tests are synchronized with your application's data flow.